CAFÉ MANAGEMENT SYSTEM

A PROJECT
SUBMITTED TO
FAKIR MOHAN AUTONOMOUS COLLEGE
IN FILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF COMPUTER APPLICATION

SUBMITTED BY

Name: Omm Prakash Behera
Exam Roll No: 23BC039
Regd. No: 9071/2023

UNDER THE SUPERVISION OF

Name: Sangeeta Das
Designation: Asst. Professor

PLACE OF RESEARCH

Department of BCA
Fakir Mohan Autonomous college,
Balasore

FAKIR MOHAN AUTONOMOUS COLLEGE,
BALASORE
2026

CAFÉ MANAGEMENT SYSTEM

A PROJECT
SUBMITTED TO
FAKIR MOHAN AUTONOMOUS COLLEGE
IN FILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF COMPUTER APPLICATION

SUBMITTED BY

Name: Omm Prakash Behera
Exam Roll No: 23BC039
Regd. No: 9071/2023

UNDER THE SUPERVISION OF

Name: Sangeeta Das
Designation: Asst. Professor

PLACE OF RESEARCH

Department of BCA
Fakir Mohan Autonomous college,
Balasore



FAKIR MOHAN AUTONOMOUS COLLEGE,
BALASORE
2026

DECLARATION

I do hereby declare that the research work presented in this project entitled **"CAFE MANAGEMENT SYSTEM"** is original and was carried out by me under the supervision of Miss Sangeeta Das, Asst. Professor Department of BCA, at Fakir Mohan Autonomous College, Balasore.

The work is original and has not been submitted for any other degree of this college or any other college or university.

Signature of the Candidate
Exam. Roll No:23BC039
Regd. No:9071/2023

Signature of Guide

Signature of the head of the Department
Department of BCA
F.M Autonomous College, Balasore

CERTIFICATE

This is to certify that the project entitled "**CAFÉ MANAGEMENT SYSTEM**" Which is being submitted by **Omm Prakash Behera(Exam Roll No:23BC039,Regd.No:9071/2023)** in fulfilment for award of BCA Degree to Department of BCA, Fakir Mohan Autonomous College, Balasore, is a record of Bonafide work carried out by her under my guidance and supervision.

The project has not been submitted to any other college or university for any other degree.

Signature of the supervisor
Name: Sangeeta Das
Designation: Asst. Professor
Fakir Mohan Autonomous College, Balasore

ACKNOWLEDGEMENT

The satisfaction that companies that the successful completion of any task would incomplete without the mention of people whose ceaseless cooperation made it possible, whose guidance and encouragement crown all efforts with success.

I am grateful to my project guide Miss Sangeeta Das guidance, inspiration and constructive suggestion that helped me in the preparation of the project.

I also thank our colleagues who have helped in the successful completion of the project.

Signature of the Candidate
Exam Roll No:23BC039
Regd. No:9071/2023

ABSTRACT

The **Cafe Management System** is a software application designed to automate and simplify the daily operations of a cafe. The main purpose of this project is to replace traditional manual processes with a digital system that improves efficiency, accuracy, and speed in managing cafe activities.

In many small cafes, tasks such as taking orders, calculating bills, and maintaining records are performed manually. This often leads to errors in billing, delays in service, and difficulty in maintaining proper records. The proposed system solves these problems by providing an automated platform where staff can easily take orders, generate bills instantly, and store data securely.

The system includes key functionalities such as user login authentication for both admin and staff, order management, billing system, and report generation. It ensures that all operations are handled smoothly with minimal human intervention, thereby reducing errors and saving time.

This project is developed using modern technologies such as React.js and TypeScript for the frontend, and Node.js for the backend. A database is used to store order and transaction details, ensuring proper data management and retrieval.

Overall, the Cafe Management System enhances the performance of cafe operations by providing a user-friendly interface and efficient workflow, making it a reliable solution for small to medium-sized cafe businesses.

TABLE OF CONTENT

CHAPTER 1: INTRODUCTION

CHAPTER 2: OBJECTIVES

CHEPTER 3: SCOPE OF THE PROJECT

CHEPTER 4: SYSTEM REQUIREMENT

CHEPTER 5: SYSTEM ARCHITECTURE

5.1: Use Case Diagram

5.2: Data Flow Diagram (DFD) Level 0

5.3: Data Flow Diagram (DFD) Level 1

5.4: ER Diagram

5.5: System Architecture

CHEPTER 6: MODULES OF THE SYSTEM

6.1: Login Module

6.2: Order Module

6.3: Billing Module

6.4: Report Module

CHEPTER 7: IMPLEMENTATION (WITH CODE)

7.1: Login Module

7.2: Order Module

7.3: Billing Module

7.4: Report Module

CHEPTER 8: SCREENSHOTS

8.1 Login Module

8.2 Dashboard

8.3 Order Screen

8.4 Billing/Receipt Screen

CHEPTER 9: ADVANTAGES

CHEPTER 10: LIMITATION

CHEPTER 11: FEATURE ENHANCEMENT

CHEPTER 12: APPENDIX

12.1 Data Structure & Interface

12.2 Core Business Logic – Order Management

12.3 Backend Server & API Configuration

12.4 Administrative Dashboard & Reports

CHEPTER 13: BIBLIOGRAPHY

1.INTRODUCTION

The Cafe Management System is a web-based application designed to automate and simplify the daily operations of a cafe. In traditional cafes, most tasks such as taking orders, managing menus, and handling customer records are done manually, which can be time-consuming and prone to errors. This system helps to reduce manual work by providing a digital platform to manage cafe activities efficiently.

This project allows administrators and staff to manage menu items, process customer orders, and maintain records in an organized manner. The system includes features such as user authentication, menu management, order handling, and data storage using cloud technology. By using modern web technologies like HTML, CSS, JavaScript, React (Vite), and Firebase, the system provides a fast, responsive, and user-friendly interface.

The main purpose of this project is to improve accuracy, save time, and enhance the overall productivity of cafe operations. It also helps in maintaining proper records and ensures secure data management. This system can be used in small and medium-sized cafes to manage their business effectively.

2.OBJECTIVES:

The main objectives of the Cafe Management System project are as follows:

- To **automate the billing system** and reduce the need for manual calculations, ensuring faster and more accurate bill generation.
- To **manage customer orders efficiently** by providing a digital platform where staff can easily select items and create orders.
- To **reduce human errors** in order processing and billing, which are common in manual systems.

- To **generate accurate reports** of daily sales and transactions for better business analysis and decision-making.
- To **improve overall efficiency** of cafe operations by streamlining workflows and reducing delays.
- To **provide a user-friendly interface** that can be easily used by both staff and administrators without requiring advanced technical knowledge.
- To **maintain proper records** of orders and transactions for future reference and tracking.
- To **enhance customer service** by reducing waiting time and ensuring quick order processing.

3.SCOPE OF THE PROJECT

The scope of the **Cafe Management System** project includes the development of a web-based application that automates the major activities involved in managing a cafe. The system is designed to handle multiple operations such as menu management, order processing, and user authentication in an efficient and organized manner. It focuses on improving the overall workflow of cafe operations and reducing the need for manual record keeping.

The project covers the **management of menu items**, where the administrator can add new items, update existing items, and remove items that are no longer available. This ensures that customers and staff always have access to an updated menu list. The system also allows staff members to view menu items and place customer orders quickly, which improves the speed of service and reduces waiting time.

Another important scope of the project is **order management**, where staff can record customer orders digitally. The system stores order

details in a database, making it easy to retrieve previous records whenever needed. This helps in maintaining proper documentation and supports better monitoring of daily sales and order activities.

The project also includes **user authentication and security features**, allowing only authorized users such as administrators and staff to access the system. This helps protect sensitive information and ensures secure operation of the system. The use of a **cloud-based database such as Firebase** expands the scope further by enabling secure storage, real-time updates, and easy data access from different devices.

The system is designed with a **user-friendly and responsive interface**, allowing it to be used on desktops, laptops, and mobile devices. This increases the usability of the system and ensures that cafe staff can operate it easily without requiring advanced technical knowledge.

In addition to the current features, the scope of this project can be extended in the future to include advanced functionalities such as **online payment integration, billing and invoice generation, customer feedback management, sales report generation, and mobile application support**. These enhancements will make the system more powerful and suitable for large-scale cafe operations.

Overall, the scope of the Cafe Management System project is broad and flexible, covering essential cafe management tasks while also allowing future improvements and scalability based on business needs.

4. SYSTEM REQUIREMENT

➢ Hardware Requirements

- Computer / Laptop:

A standard computer or laptop is required to run the application. It should be capable of handling basic development and execution tasks.

- Minimum 4GB RAM:

At least 4GB of RAM is recommended to ensure smooth performance of the system, especially while running frontend and backend simultaneously.

- Processor:

A dual-core processor or higher is preferred for better performance and faster processing.

- Storage:

Minimum 500MB of free disk space is required to store the application files and database.

➢ Software Requirements

- Node.js:

Node.js is used as the backend runtime environment to handle server-side operations and API requests.

- React.js:

React is used for developing the frontend user interface, providing a responsive and interactive experience.

- TypeScript:

TypeScript is used along with React to improve code quality, maintainability, and error handling.

- Web Browser (Google Chrome / Microsoft Edge):

A modern web browser is required to run and test the application.

- Code Editor (e.g., Visual Studio Code):

Used for writing and managing the source code efficiently.

5.SYSTEM ARCHITECTURE

➢ Architecture

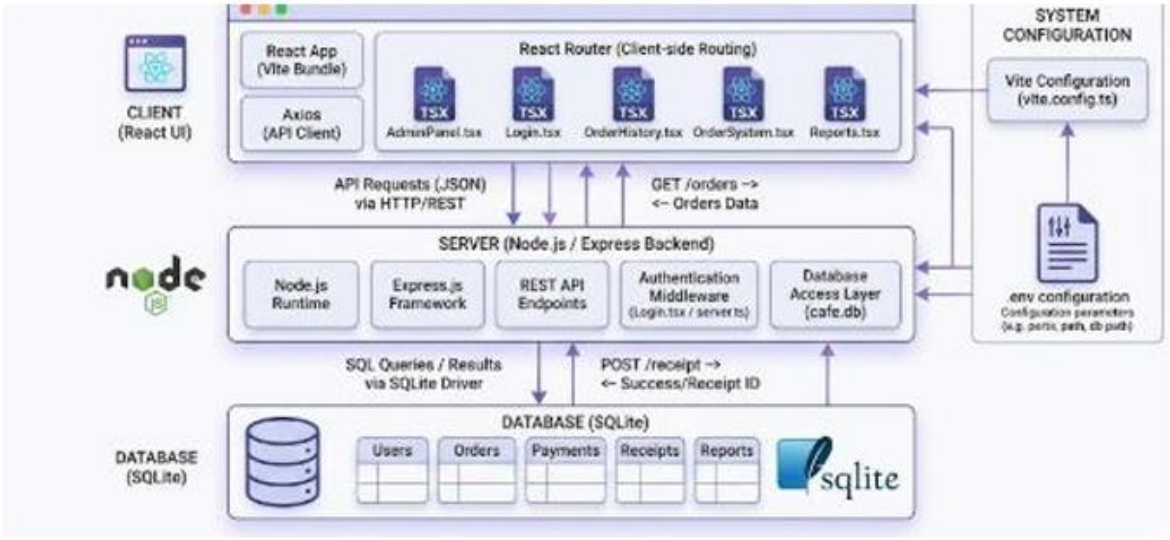
The Cafe Management System follows a client-server architecture, where the frontend, backend, and database work together to process user requests and manage data.

- Frontend (React.js):
  The frontend is developed using React.js and TypeScript. It provides a user-friendly interface where staff and admin can interact with the system. It handles user inputs such as login, order selection, and billing operations.
- Backend (Node.js):
  The backend is built using Node.js. It handles business logic, processes API requests, and communicates with the database. It also manages authentication and data validation.
- Database (SQLite):
  SQLite is used as the database to store information such as orders, users, and transactions. It ensures secure storage and quick retrieval of data.

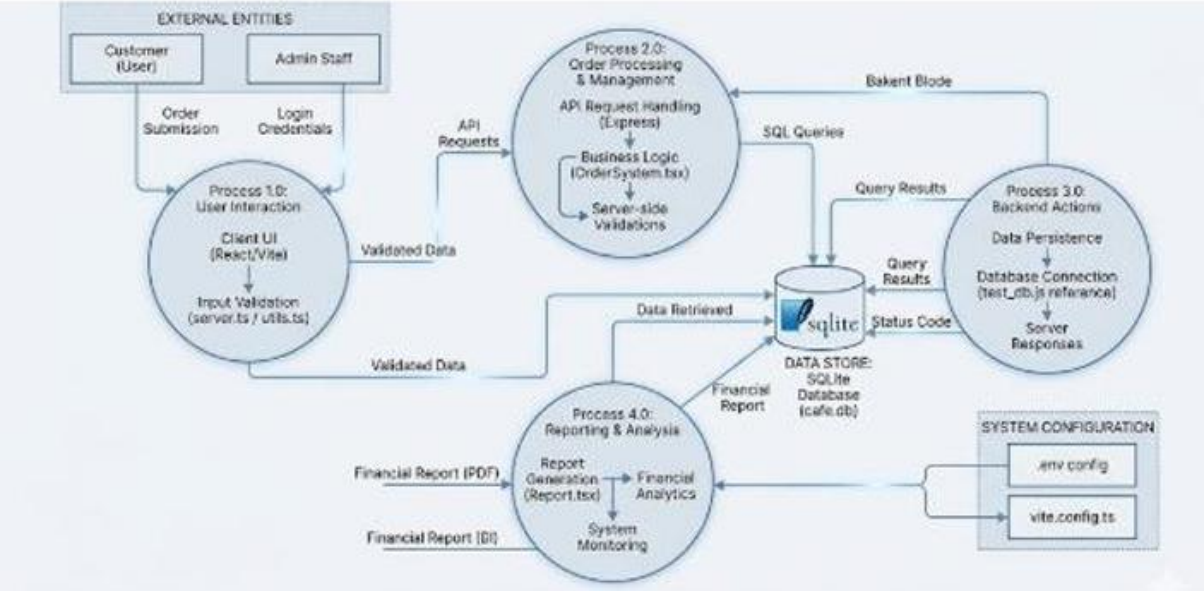
➢ Flow of the System

- User Login:
  The user (admin or staff) logs into the system using valid credentials.
- Item Selection:
  After login, the user selects items from the menu and adds them to the order.
- Bill Generation:
  The system calculates the total amount based on selected items and generates a bill.
- Data Storage:
  All order and transaction details are stored in the database for future reference.
- Report Generation:
  The system generates reports based on stored data, helping in analyzing sales and performance.

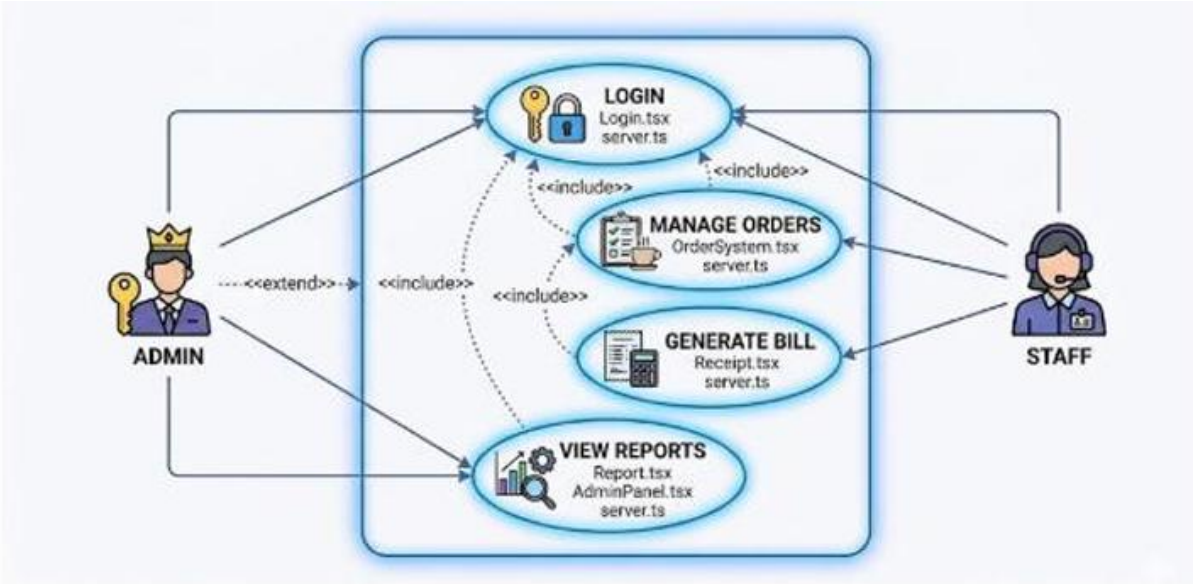
51: SYSTEM ARCHITECTURE DIAGRAM



5.2: Data Flow Diagram (DFD)



5.4: USER CASE DIAGRAM



6.MODULES OF THE SYSTEM

The Cafe Management System is divided into different modules, each responsible for handling a specific functionality. These modules work together to ensure smooth and efficient operation of the system.

6.1: **Login Module**

The Login Module is the entry point of the system. It provides authentication for both Admin and Staff users. This module ensures that only authorized users can access the system.

- The Admin login requires a username and password, which are verified through the backend server.
- The Staff login allows employees to access the system using basic credentials such as their name.
- The system checks user credentials and grants access based on the user role.
- Proper error messages are displayed in case of invalid login attempts.

6.2: Order Module

The Order Module is responsible for managing customer orders in the cafe. It allows staff to select items and create orders efficiently.

- Staff can select items from the menu and add them to the order list.
- The system allows updating item quantity or removing items if needed.
- All selected items are stored temporarily as an active order.
- The module ensures quick and accurate order processing.

6.3: Billing Module

The Billing Module handles the generation of customer bills and ensures accurate calculation of total amounts.

- It automatically calculates the total cost based on selected items and their quantities.
- A receipt is generated displaying item details, quantity, and total price.
- The system ensures error-free billing and faster checkout.
- Bills can be displayed on screen and optionally printed.

6.4: Report Module

The Report Module provides insights into the cafe's performance by analyzing sales data.

- It generates sales reports based on daily or overall transactions.
- The system calculates total revenue and number of orders.
- Data is presented in a simple format for easy understanding.
- Helps the admin in making better business decisions.

7.IMPLEMENTATION (WITH CODE)

7.1: Login Module

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setLoading(true);
  setError("");

  try {
    if (loginType === "admin") {
      const res = await fetch("/api/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username, password }),
      });
      const data = await res.json();

      if (data.success && data.user.role === "admin") {
        onLogin(data.user);
      } else {
        setError("Invalid credentials");
      }
    } else {
      const res = await fetch("/api/staff-login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username: staffName }),
      });
      const data = await res.json();

      if (data.success) {
        onLogin(data.user);
      } else {
        setError("Login failed");
      }
    }
```

```
  } catch (error) {
    setError("Server error");
  } finally {
    setLoading(false);
  }
};
```

Explanation:

This function handles both admin and staff login. It sends user credentials to the backend API and verifies the response. If authentication is successful, the user is logged into the system; otherwise, an error message is displayed.

7.2: Order Module

```
const addToCart = (item: MenuItem) => {
  setCartItems(prev => {
    const existing = prev.find(c => c.menuItem.id === item.id);
    if (existing) {
      return prev.map(c =>
        c.menuItem.id === item.id ? { ...c, qty: c.qty + 1 } : c
      );
    }
    return [...prev, { menuItem: item, qty: 1 }];
  });
};
```

Explanation:

This function allows items to be added to the cart. If the item already exists, its quantity is increased; otherwise, it is added as a new item. This helps in managing orders efficiently.

7.3 Billing Module

```
<div>
  {items && items.length > 0 ? (
    items.map((i, idx) => (
      <div key={idx}>
        {i.name} - {i.qty} x ₹{i.price.toFixed(2)}
      </div>
    ))
```

```
  ) : (
    <div>
      {item} - {qty} x ₹{(total / (qty || 1)).toFixed(2)}
    </div>
  )}

  <div>
    <strong>Total: ₹{total.toFixed(2)}</strong>
  </div>
</div>
```

Explamation:

This code is used to generate the customer receipt in the billing module. It displays all ordered items using the map() function, showing their name, quantity, and price. If only a single item is present, it is displayed separately. Finally, the total amount is calculated and shown at the bottom of the receipt, ensuring clear and accurate billing.

7.4: Report Module

```
// Fetch Report Data from Backend
const fetchReport = async () => {
  const res = await fetch(`/api/report?date=${date}`);
  const data = await res.json();
  setReport(data);
};

// Sales Calculation
const totalSales = report?.totalSales || 0;
const totalOrders = report?.totalOrders || 0;
```

Explanation

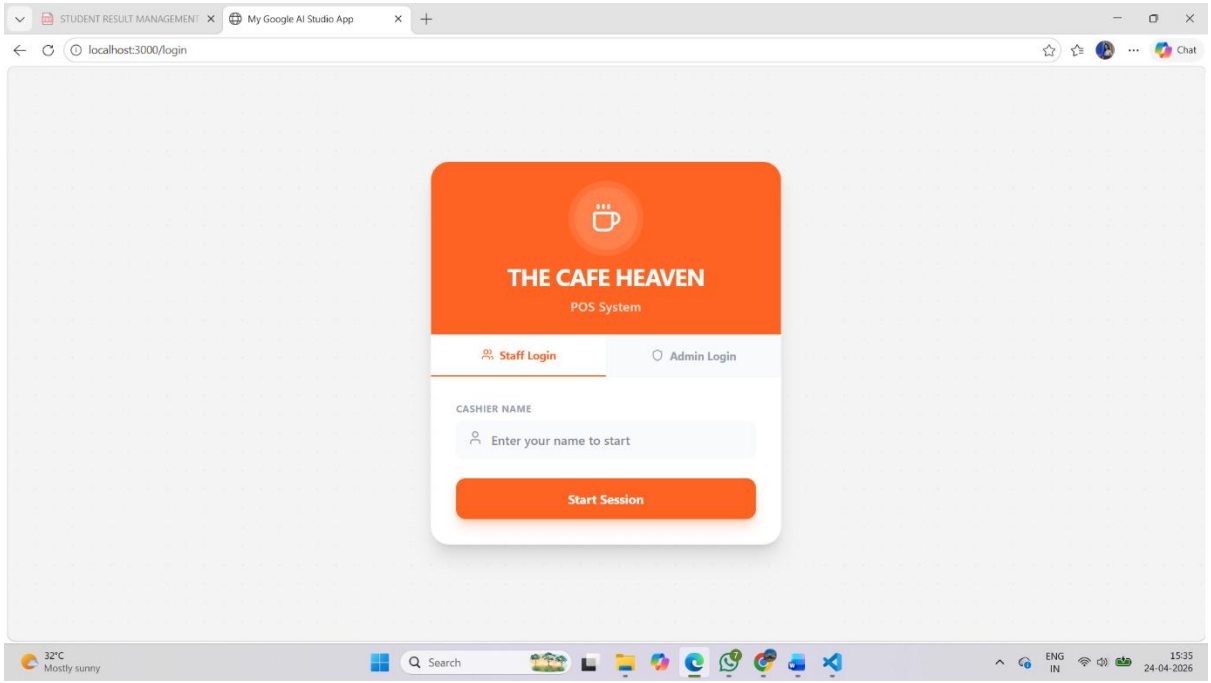
This code is used to generate reports in the Cafe Management System. The fetchReport function retrieves sales data from the backend API based on the selected date. The totalSales variable stores the total revenue, and totalOrders represents the total number of orders. This information helps in analyzing daily performance and business growth.

8. SCREENSHOTS

This section includes screenshots of the Cafe Management System to demonstrate the user interface and functionality of different modules. These screenshots help in understanding how the system works in real-time.

8.1: Login Page

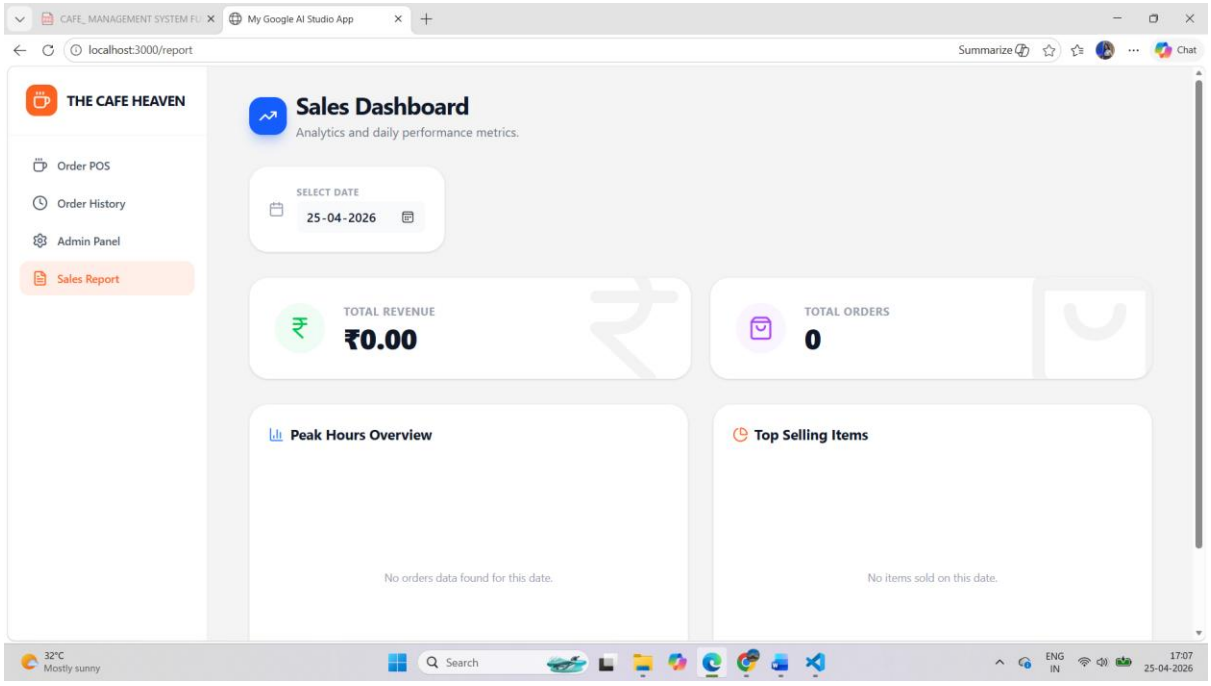
The login page allows both admin and staff to access the system by entering their credentials. It ensures secure authentication before granting access.



"Figure 8.1: Login Page"

8.2: Dashboard

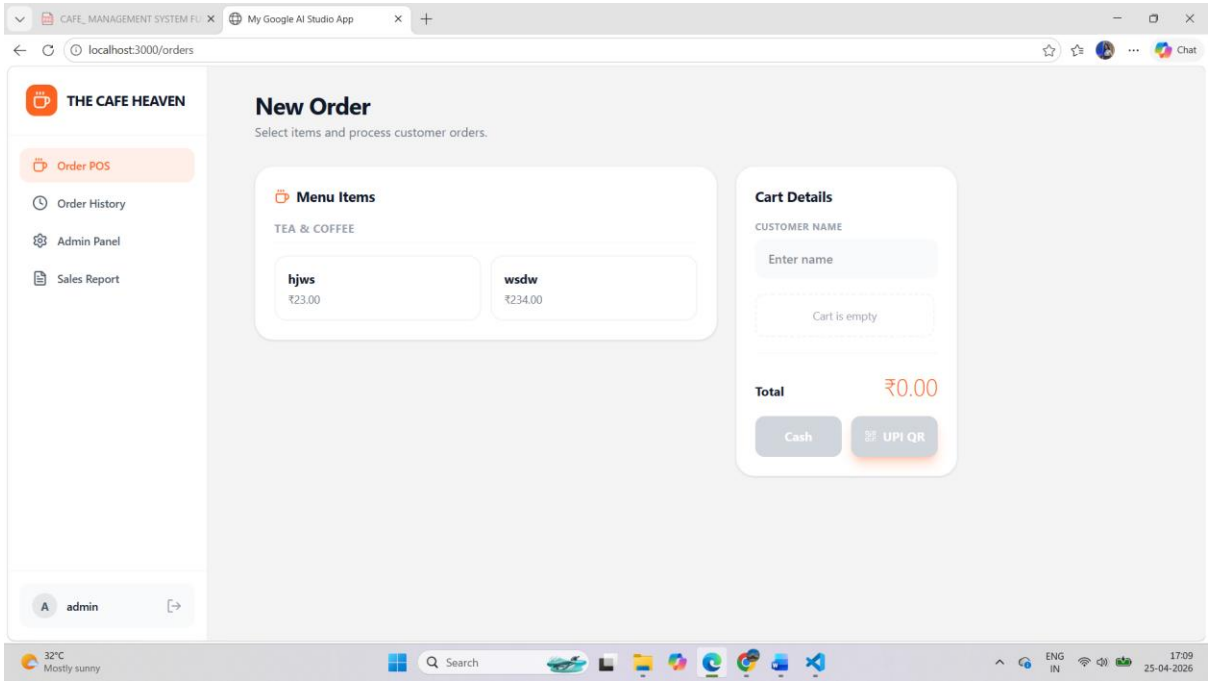
The dashboard provides an overview of the system, including navigation to different modules such as orders, reports, and billing. It acts as the main control panel for users.



"Figure 8.2: Dashboard Page"

8.3: Order Screen

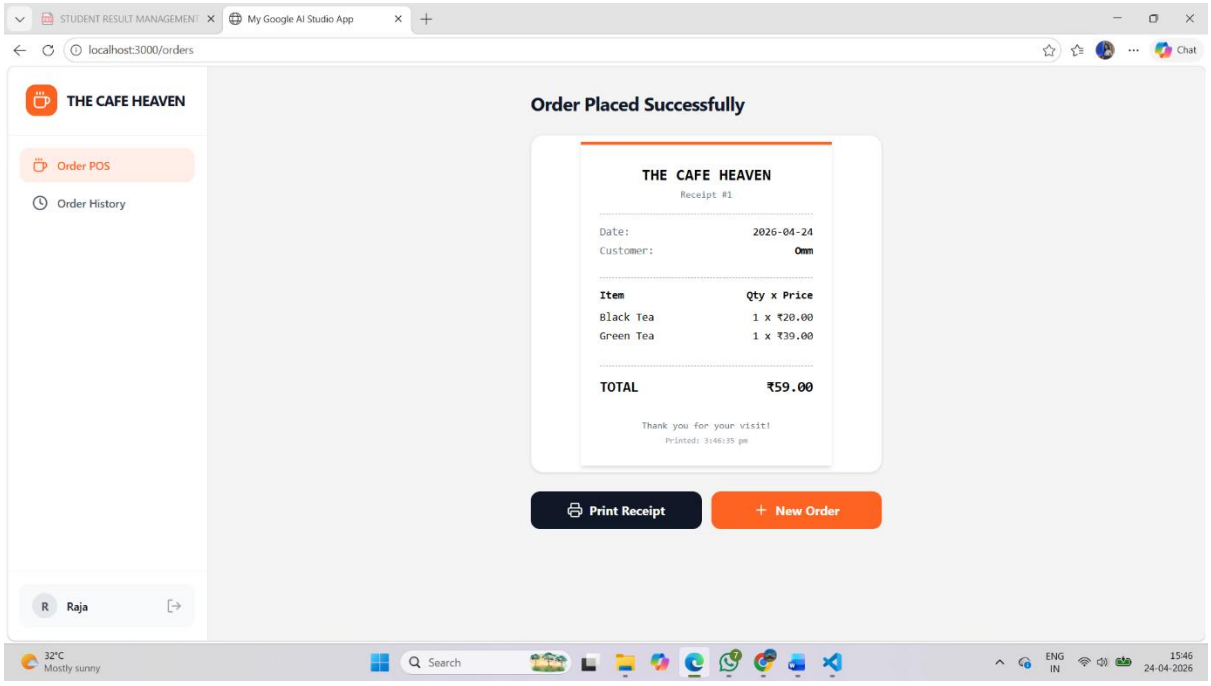
The order screen allows staff to select menu items, add them to the cart, and manage customer orders efficiently. It shows item details, quantity, and total amount.



"Figure 8.3: Order Page"

8.4: Billing/Receipt Screen

The billing screen displays the generated receipt, including item details, quantity, price, and total amount. It ensures accurate billing for customers.



"Figure 8.4: Billing Page"

9. ADVANTAGES

The Cafe Management System offers several benefits that improve the overall efficiency and performance of cafe operations:

- **Saves Time:**
  The system automates order taking and billing processes, reducing the time required to serve customers and improving service speed.
- **Accurate Billing:**
  All calculations are performed automatically by the system, ensuring error-free billing and eliminating mistakes caused by manual calculations.
- **Easy to Use:**
  The system provides a simple and user-friendly interface, allowing staff to operate it easily without requiring advanced technical knowledge.
- **Reduces Human Error:**
  By automating tasks such as order management and billing, the system minimizes the chances of human errors.
- **Improves Efficiency:**
  Streamlines the workflow of the cafe, making operations faster and more organized.
- **Better Record Management:**
  All order and transaction data are stored digitally, making it easy to access and manage records.
- **Enhances Customer Experience:**
  Faster service and accurate billing lead to improved customer satisfaction.

10.LIMITATION

Although the Cafe Management System provides many advantages, it also has certain limitations:

- **Internet Required (if online system is used):**
  The system may require a stable internet connection for data fetching and backend communication. Without internet, some features may not work properly.

- **Limited Features:**
  The current version of the system includes only basic functionalities such as ordering, billing, and reporting. Advanced features like inventory management and online delivery are not included.
- **Not Scalable for Large Businesses:**
  This system is mainly designed for small to medium cafes. It may not perform efficiently for large-scale restaurant chains with high traffic and complex operations.
- **Dependency on System Performance:**
  The performance of the application depends on the system hardware and server response time.
- **No Advanced Security Features:**
  The system uses basic authentication and does not include advanced security mechanisms like multi-factor authentication.

11. FEATURE ENHANCEMENT

The Cafe Management System can be further improved by adding new features and technologies to enhance its functionality and user experience. These enhancements will make the system more advanced, scalable, and suitable for modern business needs. One of the major future improvements is the addition of an online ordering system. This will allow customers to place orders remotely through a website, reducing in-store crowd and improving convenience for users. Another important enhancement is the development of a mobile application. A dedicated mobile app will enable both customers and cafe staff to access the system anytime and from anywhere, making operations more flexible and efficient. The system can also be upgraded by integrating a cloud database. Moving data storage to the cloud will improve data security, allow real time access from multiple locations, and ensure better scalability as the business grows. In addition, future upgrades may include features like online payment integration, real-time notifications, and advanced analytics for better decision-making.

12.APPENDIX

12.1: Data Structures & Interfaces (src/types.ts)

Description: This file defines the core data models for the application using TypeScript interfaces. It ensures data consistency across the frontend and backend modules.

```
export interface User {
  id: number;
  username: string;
  role: string;
}

export interface MenuItem {
  id: number;
  name: string;
  price: number;
  category?: string;
}

export interface OrderItem {
  name: string;
  qty: number;
  price: number;
}

export interface Order {
  id: number;
  customer: string;
  item: string;
  qty: number;
  total: number;
  date: string;
  status?: string;
  created_at?: string;
  items?: OrderItem[];
}
```

```
export interface ReportData {
  totalSales: number | null;
  totalOrders: number | null;
  topItems?: { name: string, value: number }[];
  peakHours?: { time: string, orders: number }[];
}
```

12.2: Core Business Logic-Order management (src/components/OrderSystem.tsx)

Description: This is the main functional component that handles the shopping cart logic, real-time price calculation, and order placement process.

```
import React, { useState, useEffect } from "react";
import { MenuItem } from "../types";
import { Coffee, Printer, Plus, Minus, Send, QrCode, Trash2 } from "lucide-react";
import { QRCodeSVG } from "qrcode.react";
import Receipt from "./Receipt";

interface CartItem {
  menuItem: MenuItem;
  qty: number;
}

export default function OrderSystem() {
  const [customer, setCustomer] = useState("");
  const [menuItems, setMenuItems] = useState<MenuItem[]>([]);
  const [cartItems, setCartItems] = useState<CartItem[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  // Receipt & Payment State
  const [showReceipt, setShowReceipt] = useState(false);
  const [showPayment, setShowPayment] = useState(false);
  const [lastOrder, setLastOrder] = useState<any>(null);
```

```
// Default UPI ID for demo
const UPI_ID = "merchant@upi";

useEffect(() => {
  fetch("/api/menu")
    .then(res => res.json())
    .then(data => {
      setMenuItems(data);
    })
    .catch(err => console.error(err));
}, []);

    const total = cartItems.reduce((acc, item) => acc + (item.menuItem.price * item.qty), 0);
                        const upiLink = `upi://pay?pa=${UPI_ID}&pn=The+Cafe+Heaven&am=${total.toFixed(2)}&cu=INR`;

const addToCart = (item: MenuItem) => {
  setCartItems(prev => {
    const existing = prev.find(c => c.menuItem.id === item.id);
    if (existing) {
      return prev.map(c => c.menuItem.id === item.id ? { ...c, qty: c.qty + 1 } : c);
    }
    return [...prev, { menuItem: item, qty: 1 }];
  });
};

const updateCartQty = (id: number, delta: number) => {
  setCartItems(prev => prev.map(c => {
    if (c.menuItem.id === id) {
      return { ...c, qty: Math.max(1, c.qty + delta) };
    }
```

```
    return c;
  }));
};

const removeFromCart = (id: number) => {
  setCartItems(prev => prev.filter(c => c.menuItem.id !== id));
};

const handlePlaceOrder = async () => {
  if (cartItems.length === 0 || !customer) return;

  setLoading(true);
  setError("");

  const date = new Date().toISOString().split('T')[0];

  const orderData = {
    customer,
    items: cartItems.map(c => ({
      name: c.menuItem.name,
      price: c.menuItem.price,
      qty: c.qty
    })),
    total,
    date
  };

  try {
    const res = await fetch("/api/orders", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(orderData),
    });
    const data = await res.json();
```

```
    if (data.success) {
      setLastOrder({ ...orderData, orderId: data.orderId });
      setShowPayment(false);
      setShowReceipt(true);
      setCustomer("");
      setCartItems([]);
    } else {
      setError(data.message || "Failed to place order");
    }
  } catch (err) {
    setError("Server error");
  } finally {
    setLoading(false);
  }
};

const initPayment = (e: React.FormEvent) => {
  e.preventDefault();
  if (cartItems.length === 0 || !customer) return;
  setShowPayment(true);
};

const printReceipt = () => {
      const printContent = document.getElementById("receipt-printarea");
  const windowPrint = window.open('', '', 'width=400,height=600');
  if (windowPrint && printContent) {
            windowPrint.document.write('<html><head><title>Print Receipt</title>');
                        windowPrint.document.write('<script src="https://cdn.tailwindcss.com"></script>');
    windowPrint.document.write('</head><body class="p-4">');
    windowPrint.document.write(printContent.outerHTML);
    windowPrint.document.write('</body></html>');
    windowPrint.document.close();
```

```
    windowPrint.focus();
    setTimeout(() => {
      windowPrint.print();
      windowPrint.close();
    }, 500);
  }
};

 if (showReceipt && lastOrder) {
  return (
    <div className="max-w-md mx-auto space-y-6">
      <h2 className="text-2xl font-bold tracking-tight">Order Placed Successfully</h2>
       <div className="bg-white p-2 rounded-2xl shadow-sm border border-gray-100">
       <Receipt {...lastOrder} />
      </div>
      <div className="flex space-x-3">
       <button
        onClick={printReceipt}
         className="flex-1 bg-gray-900 text-white font-bold py-3 px-4 rounded-xl flex items-center justify-center space-x-2 hover:bg-gray-800 transition-colors"
       >
        <Printer className="w-5 h-5" />
        <span>Print Receipt</span>
       </button>
       <button
        onClick={() => setShowReceipt(false)}
        className="flex-1 bg-[#FF6321] text-white font-bold py-3 px-4 rounded-xl flex items-center justify-center space-x-2 hover:bg-[#e5581e] transition-colors"
       >
        <Plus className="w-5 h-5" />
        <span>New Order</span>
```

```
        </button>
      </div>
    </div>
  );
 }

 return (
  <div className="max-w-4xl">
   <div className="mb-8">
        <h2 className="text-3xl font-bold tracking-tight text-gray-900">New Order</h2>
       <p className="text-gray-500 mt-1">Select items and process customer orders.</p>
   </div>

   <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
     <div className="md:col-span-1 lg:col-span-2 space-y-6">
         <div className="bg-white rounded-3xl shadow-sm border border-gray-100 p-6">
       <h3 className="text-lg font-bold mb-4 flex items-center space-x-2">
         <Coffee className="w-5 h-5 text-[#FF6321]" />
         <span>Menu Items</span>
        </h3>

        <div className="space-y-8">
         {Object.entries(
          menuItems.reduce((acc, item) => {
           const cat = item.category || 'Uncategorized';
           if (!acc[cat]) acc[cat] = [];
           acc[cat].push(item);
           return acc;
          }, {} as Record<string, MenuItem[]>)
         ).map(([category, items]) => (
```

```
            <div key={category}>
              <h4 className="text-sm font-bold text-gray-400 uppercase
tracking-wider mb-4 border-b border-gray-100 pb-2">{category}</h4>
              <div className="grid grid-cols-2 gap-3">
                {items.map(item => (
                  <button
                    key={item.id}
                    type="button"
                    onClick={() => addToCart(item)}
                   className="p-4 rounded-2xl border-2 text-left transition-
all duration-200 border-gray-100 bg-white hover:border-[#FF6321]
active:scale-95"
                   >
                                  <div  className="font-bold  text-gray-900
truncate">{item.name}</div>
                               <div  className="text-gray-500  text-sm  mt-
1">₹{item.price.toFixed(2)}</div>
                  </button>
                ))}
              </div>
            </div>
          ))}
          {menuItems.length === 0 && (
            <div className="text-center text-gray-500 py-8 text-sm">
              No items in menu. Add some from Admin Panel.
            </div>
          )}
        </div>
      </div>
    </div>

    <div className="space-y-6">
        <div  className="bg-white  rounded-3xl  shadow-sm  border
border-gray-100 p-6 sticky top-8">
      <h3 className="text-lg font-bold mb-4">Cart Details</h3>
```

```
        <form onSubmit={initPayment} className="space-y-5">
                    {error && <div className="text-red-500 text-sm">{error}</div>}

          <div>
              <label className="block text-xs font-bold text-gray-400 uppercase tracking-wider mb-2">Customer Name</label>
            <input
              type="text"
              value={customer}
              onChange={(e) => setCustomer(e.target.value)}
                 className="w-full px-4 py-3 bg-gray-50 border border-transparent focus:border-gray-300 focus:bg-white rounded-xl outline-none transition-all font-medium"
              placeholder="Enter name"
              required
            />
          </div>

          <div className="space-y-3 max-h-[300px] overflow-y-auto pr-1">
            {cartItems.length === 0 ? (
                <div className="text-center text-gray-400 text-sm py-4 border-2 border-dashed border-gray-100 rounded-xl">Cart is empty</div>
            ) : (
              cartItems.map((cartItem) => (
                <div key={cartItem.menuItem.id} className="flex flex-col bg-gray-50 p-3 rounded-xl border border-gray-100">
                  <div className="flex justify-between items-start mb-2">
                            <div className="font-bold text-sm text-gray-900">{cartItem.menuItem.name}</div>
                                <div className="font-medium text-sm">₹{(cartItem.menuItem.price * cartItem.qty).toFixed(2)}</div>
```

```
            </div>
            <div className="flex justify-between items-center">
              <div className="flex items-center space-x-2 bg-white rounded-lg border border-gray-200">
                <button type="button" onClick={() => updateCartQty(cartItem.menuItem.id, -1)} className="w-7 h-7 flex items-center justify-center text-gray-600 hover:bg-gray-100 rounded-l-lg hover:text-black">
                  <Minus className="w-3 h-3" />
                </button>
                <span className="w-4 text-center font-bold text-sm">{cartItem.qty}</span>
                <button type="button" onClick={() => updateCartQty(cartItem.menuItem.id, 1)} className="w-7 h-7 flex items-center justify-center text-gray-600 hover:bg-gray-100 rounded-r-lg hover:text-black">
                  <Plus className="w-3 h-3" />
                </button>
              </div>
              <button type="button" onClick={() => removeFromCart(cartItem.menuItem.id)} className="text-red-400 hover:text-red-600 p-1">
                <Trash2 className="w-4 h-4" />
              </button>
            </div>
          </div>
        ))
      )}
    </div>

    <div className="pt-4 border-t border-gray-100">
      <div className="flex justify-between items-end mt-2">
        <span className="text-gray-900 font-bold">Total</span>
        <span className="text-3xl font-light text-[#FF6321]">₹{total.toFixed(2)}</span>
```

```
            </div>
          </div>

          <div className="grid grid-cols-2 gap-3 mt-4">
            <button
              type="button"
              onClick={handlePlaceOrder}
              disabled={cartItems.length === 0 || !customer || loading}
                        className="w-full bg-gray-900 hover:bg-gray-800
disabled:bg-gray-300 text-white font-bold py-3.5 px-4 rounded-xl
transition-all active:scale-[0.98] flex items-center justify-center space-
x-2"
            >
              {loading ? (
               <div className="w-5 h-5 border-2 border-white/30 border-
t-white rounded-full animate-spin" />
              ) : (
                <span>Cash</span>
              )}
            </button>
            <button
              type="submit"
              disabled={cartItems.length === 0 || !customer || loading}
                      className="w-full bg-[#FF6321] hover:bg-[#e5581e]
disabled:bg-gray-300 text-white font-bold py-3.5 px-4 rounded-xl
transition-all active:scale-[0.98] shadow-lg shadow-[#FF6321]/30 flex
items-center justify-center space-x-2"
            >
              <QrCode className="w-4 h-4" />
              <span>UPI QR</span>
            </button>
          </div>
        </form>
      </div>
    </div>
```

```
    </div>

    {showPayment && (
     <div className="fixed inset-0 bg-black/50 backdrop-blur-sm z-50 flex items-center justify-center p-4">
        <div className="bg-white p-8 rounded-3xl shadow-2xl text-center max-w-sm w-full relative border border-gray-100">
        <button
         onClick={() => setShowPayment(false)}
          className="absolute top-4 right-4 text-gray-400 hover:text-gray-900 w-8 h-8 flex items-center justify-center rounded-full bg-gray-100 hover:bg-gray-200"
        >
         ×
        </button>
         <div className="w-16 h-16 bg-blue-50 rounded-full mx-auto flex items-center justify-center mb-4">
         <QrCode className="w-8 h-8 text-blue-500" />
        </div>
       <h3 className="text-2xl font-bold tracking-tight mb-2">Scan to Pay</h3>
          <p className="text-gray-500 mb-6 font-medium">Amount: <span className="text-[#FF6321] font-bold text-xl">₹{total.toFixed(2)}</span></p>

         <div className="bg-white p-4 rounded-2xl border-2 border-gray-100 inline-block shadow-sm mb-8">
         <QRCodeSVG value={upiLink} size={220} level="H" />
        </div>

        <p className="text-xs text-gray-400 mb-6 px-4">
           Ensure customer has completed the payment successfully before confirming the order.
        </p>
```

```
          <button
            onClick={handlePlaceOrder}
            disabled={loading}
           className="block w-full bg-[#10B981] text-white font-bold py-4 px-4 rounded-xl hover:bg-[#059669] transition-all active:scale-95 shadow-lg shadow-[#10B981]/30 flex justify-center items-center h-14"
          >
            {loading ? (
              <div className="w-6 h-6 border-2 border-white/30 border-t-white rounded-full animate-spin" />
            ) : (
              <span className="text-lg">Payment Received</span>
            )}
          </button>
        </div>
      </div>
    )}
  </div>
 );
}
```

12.3: Backend Server & API Configuration (server.ts)

Description: This file contains the Node.js/Express server setup. It manages the API endpoints and handles communication between the client-side and the database.

```
 import express from 'express';
import { createServer as createViteServer } from 'vite';
import path from 'path';
import Database from 'better-sqlite3';

async function startServer() {
  const app = express();
  const PORT = 3000;

  app.use(express.json());
```

```
// Database setup
const dbPath = path.join(process.cwd(), 'cafe.db');
const db = new Database(dbPath);

// Initialize DB tables
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE,
    password TEXT,
    role TEXT
  );
  CREATE TABLE IF NOT EXISTS menu (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT UNIQUE,
    price REAL,
    category TEXT DEFAULT 'Uncategorized'
  );
  CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer TEXT,
    item TEXT,
    qty INTEGER,
    total REAL,
    date TEXT,
    status TEXT DEFAULT 'pending',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
  );
  CREATE TABLE IF NOT EXISTS order_items (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER,
    item_name TEXT,
    qty INTEGER,
    price REAL,
```

```
    FOREIGN KEY(order_id) REFERENCES orders(id)
  );
`);

// Add columns to existing table if missing migration
try {
  db.exec(`ALTER TABLE menu ADD COLUMN category TEXT DEFAULT 'Uncategorized'`);
} catch (e) {}
try {
  db.exec(`ALTER TABLE orders ADD COLUMN status TEXT DEFAULT 'pending'`);
} catch (e) {}
try {
  // SQLite doesn't support non-constant DEFAULT in ALTER TABLE ADD COLUMN
  db.exec(`ALTER TABLE orders ADD COLUMN created_at TEXT`);
  // Backfill existing rows
  db.exec(`UPDATE orders SET created_at = date || ' 12:00:00' WHERE created_at IS NULL`);
} catch (e) {}

try {
  const checkTable = db.prepare("SELECT name FROM sqlite_master WHERE type='table' AND name='order_items'").get();
  if (!checkTable) {
    db.exec(`
      CREATE TABLE order_items (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        order_id INTEGER,
        item_name TEXT,
        qty INTEGER,
        price REAL,
        FOREIGN KEY(order_id) REFERENCES orders(id)
      );
```

```
      `);
    }

     const count = db.prepare('SELECT COUNT(*) as count FROM order_items').get() as {count: number};
    if (count.count === 0) {
     db.exec(`
      INSERT INTO order_items (order_id, item_name, qty, price)
      SELECT id, item, qty, total/qty FROM orders WHERE item IS NOT NULL AND qty > 0
     `);
    }
  } catch (e) {}

  // Create default admin if not exists
   const checkAdmin = db.prepare('SELECT * FROM users WHERE username = ?').get('admin');
  if (!checkAdmin) {
    db.prepare('INSERT INTO users (username, password, role) VALUES (?, ?, ?)').run('admin', 'admin123', 'admin');
  }

  // --- API Routes ---

  // Staff Login (No Password)
  app.post('/api/staff-login', (req, res) => {
    const { username } = req.body;
    if (!username) {
     return res.status(400).json({ success: false, message: 'Please enter your name' });
    }
    // Create a temporary session object for staff
    res.json({ success: true, user: { id: Date.now(), username, role: 'staff' } });
  });
```

```
// Login
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;
   const user = db.prepare('SELECT id, username, role FROM users WHERE username = ? AND password = ?').get(username, password);
  if (user) {
    res.json({ success: true, user });
  } else {
     res.status(401).json({ success: false, message: 'Invalid credentials' });
  }
});

// Get Menu
app.get('/api/menu', (req, res) => {
  const items = db.prepare('SELECT * FROM menu').all();
  res.json(items);
});

// Add Item
app.post('/api/menu', (req, res) => {
  const { name, price, category } = req.body;
  try {
    db.prepare('INSERT INTO menu (name, price, category) VALUES (?, ?, ?)').run(name, price, category || 'Uncategorized');
    res.json({ success: true });
  } catch (e: any) {
    res.status(400).json({ success: false, message: e.message });
  }
});

// Delete Item
app.delete('/api/menu/:id', (req, res) => {
  try {
```

```
            db.prepare('DELETE FROM menu WHERE id = ?').run(req.params.id);
    res.json({ success: true });
   } catch (e: any) {
    res.status(400).json({ success: false, message: e.message });
   }
 });

 // Place Order
 app.post('/api/orders', (req, res) => {
  const { customer, items, total, date } = req.body;
  try {
   const created_at = new Date().toISOString();
       const itemTitle = items.length === 1 ? items[0].name : `${items.length} Items`;
   const totalQty = items.reduce((sum: number, i: any) => sum + i.qty, 0);

   const result = db.prepare('INSERT INTO orders (customer, item, qty, total, date, status, created_at) VALUES (?, ?, ?, ?, ?, ?, ?)').run(customer, itemTitle, totalQty, total, date, 'pending', created_at);
   const orderId = result.lastInsertRowid;

   const insertItem = db.prepare('INSERT INTO order_items (order_id, item_name, qty, price) VALUES (?, ?, ?, ?)');
   for (const item of items) {
    insertItem.run(orderId, item.name, item.qty, item.price);
   }

   res.json({ success: true, orderId });
  } catch (e: any) {
   res.status(400).json({ success: false, message: e.message });
  }
 });
```

```
 // Get Single Order
 app.get('/api/orders/:id', (req, res) => {
    const order = db.prepare('SELECT * FROM orders WHERE id = ?').get(req.params.id) as any;
   if (order) {
     order.items = db.prepare('SELECT item_name as name, qty, price FROM order_items WHERE order_id = ?').all(req.params.id);
    res.json(order);
   } else {
    res.status(404).json({ message: 'Not found' });
   }
 });

 // Update Order Status
 app.put('/api/orders/:id/status', (req, res) => {
  const { status } = req.body;
  try {
       db.prepare('UPDATE orders SET status = ? WHERE id = ?').run(status, req.params.id);
    res.json({ success: true });
  } catch (e: any) {
    res.status(400).json({ success: false, message: e.message });
  }
 });

 // Get Orders (Real-time history simulator)
 app.get('/api/orders', (req, res) => {
  const orders = db.prepare('SELECT * FROM orders ORDER BY id DESC LIMIT 50').all() as any[];
   const allItems = db.prepare('SELECT * FROM order_items').all() as any[];

  orders.forEach(order => {
    order.items = allItems
```

```
      .filter(i => i.order_id === order.id)
      .map(i => ({ name: i.item_name, qty: i.qty, price: i.price }));
  });

  res.json(orders);
 });

 // Daily Sales Report
 app.get('/api/report', (req, res) => {
  const { date } = req.query; // date in YYYY-MM-DD
    const report = db.prepare('SELECT SUM(total) as totalSales, COUNT(id) as totalOrders FROM orders WHERE date = ?').get(date) as any;

    const topItems = db.prepare('SELECT item_name as name, SUM(order_items.qty) as value FROM order_items JOIN orders ON order_items.order_id = orders.id WHERE orders.date = ? GROUP BY item_name ORDER BY value DESC LIMIT 5').all(date);

  const peakHours = db.prepare(`
    SELECT strftime('%H', datetime(created_at, 'localtime')) as hour, COUNT(id) as orders
    FROM orders
    WHERE date = ?
    GROUP BY hour
    ORDER BY hour ASC
  `).all(date) as any[];

  const peakHoursData = peakHours.map(p => ({
    time: `${p.hour}:00`,
    orders: p.orders
  }));

  res.json({
    totalSales: report.totalSales || 0,
```

```
      totalOrders: report.totalOrders || 0,
      topItems: topItems || [],
      peakHours: peakHoursData || []
    });
  });

  // Vite middleware setup
  if (process.env.NODE_ENV !== "production") {
    const fs = await import('fs');
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: "spa",
    });
    app.use(vite.middlewares);

    app.use('*', async (req, res, next) => {
      try {
            let template = fs.readFileSync(path.resolve(process.cwd(), 'index.html'), 'utf-8');
            template = await vite.transformIndexHtml(req.originalUrl, template);
        res.status(200).set({ 'Content-Type': 'text/html' }).end(template);
      } catch (e) {
        vite.ssrFixStacktrace(e as Error);
        next(e);
      }
    });
  } else {
    // Note: since this is built using tsc to a cjs file, __dirname might fail or behave differently.
    // It is best to use process.cwd()
    const distPath = path.join(process.cwd(), 'dist');
    app.use(express.static(distPath));
    app.get('*', (req, res) => {
      res.sendFile(path.join(distPath, 'index.html'));
```

```
  });
 }

 app.listen(PORT, "0.0.0.0", () => {
  console.log(`Server running on http://localhost:${PORT}`);
 });
}

startServer();
```

12.4: Administrative Dashboard & Reports (src/components/Report.tsx)

Description: This module processes raw transaction data to generate sales reports, helping the administrator analyze business performance.

```
import { useState, useEffect } from "react";
import { ReportData } from "../types";
import { FileText, TrendingUp, Calendar, IndianRupee, ShoppingBag,
BarChart3, PieChart as PieChartIcon } from "lucide-react";
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer,
PieChart, Pie, Cell, Legend } from "recharts";

const COLORS = ['#FF6321', '#3B82F6', '#10B981', '#F59E0B',
'#8B5CF6'];

export default function Report() {
 const [report, setReport] = useState<ReportData | null>(null);
 const [loading, setLoading] = useState(false);
 const [date, setDate] = useState(new
Date().toISOString().split('T')[0]);

 useEffect(() => {
  fetchReport();
 }, [date]);

 const fetchReport = async () => {
```

```
    setLoading(true);
    try {
      const res = await fetch(`/api/report?date=${date}`);
      const data = await res.json();
      setReport(data);
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const totalSales = report?.totalSales || 0;
  const totalOrders = report?.totalOrders || 0;
  const peakHours = report?.peakHours || [];
  const topItems = report?.topItems || [];

  return (
    <div className="max-w-6xl w-full">
      <div className="mb-8 flex items-center space-x-3">
        <div className="w-12 h-12 bg-blue-600 rounded-2xl flex items-
center justify-center text-white shadow-lg shadow-blue-600/30">
          <TrendingUp className="w-6 h-6" />
        </div>
        <div>
          <h2 className="text-3xl font-bold tracking-tight text-gray-
900">Sales Dashboard</h2>
          <p className="text-gray-500 mt-1">Analytics and daily
performance metrics.</p>
        </div>
      </div>

      <div className="bg-white rounded-3xl shadow-sm border
border-gray-100 p-6 mb-8 inline-block">
        <div className="flex items-center space-x-4">
```

```
        <Calendar className="w-5 h-5 text-gray-400" />
        <div>
         <label className="block text-xs font-bold text-gray-400
uppercase tracking-wider mb-1">Select Date</label>
         <input
          type="date"
          value={date}
          onChange={(e) => setDate(e.target.value)}
          className="bg-gray-50 border-none rounded-lg px-3 py-2
font-medium text-gray-900 outline-none focus:ring-2 focus:ring-blue-
500/20 cursor-pointer"
         />
        </div>
       </div>
      </div>

     <div className="grid grid-cols-1 md:grid-cols-2 gap-6 mb-8">
       <div className="bg-white rounded-3xl shadow-sm border
border-gray-100 p-8 flex items-center space-x-6 relative overflow-
hidden">
        <div className="w-16 h-16 rounded-full bg-green-50 flex
items-center justify-center relative z-10">
         <IndianRupee className="w-8 h-8 text-green-500" />
        </div>
        <div className="relative z-10">
         <div className="text-sm font-bold text-gray-400 uppercase
tracking-wider mb-1">Total Revenue</div>
         <div className="text-4xl font-black tracking-tight text-gray-
900">₹{totalSales.toFixed(2)}</div>
        </div>
        <div className="absolute -right-6 -bottom-6 opacity-5">
         <IndianRupee className="w-48 h-48" />
        </div>
       </div>
```

```
      <div className="bg-white rounded-3xl shadow-sm border border-gray-100 p-8 flex items-center space-x-6 relative overflow-hidden">
        <div className="w-16 h-16 rounded-full bg-purple-50 flex items-center justify-center relative z-10">
          <ShoppingBag className="w-8 h-8 text-purple-500" />
        </div>
        <div className="relative z-10">
          <div className="text-sm font-bold text-gray-400 uppercase tracking-wider mb-1">Total Orders</div>
          <div className="text-4xl font-black tracking-tight text-gray-900">{totalOrders}</div>
        </div>
        <div className="absolute -right-6 -bottom-6 opacity-5">
          <ShoppingBag className="w-48 h-48" />
        </div>
      </div>
    </div>

    {loading ? (
      <div className="text-center text-gray-500 py-12">Loading analytics...</div>
    ) : (
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
        {/* Peak Hours Bar Chart */}
        <div className="bg-white rounded-3xl shadow-sm border border-gray-100 p-6">
          <h3 className="text-lg font-bold mb-6 flex items-center space-x-2">
            <BarChart3 className="w-5 h-5 text-blue-500" />
            <span>Peak Hours Overview</span>
          </h3>
          {peakHours.length > 0 ? (
            <div className="h-72 w-full">
              <ResponsiveContainer width="100%" height="100%">
```

```
              <BarChart data={peakHours} margin={{ top: 10, right: 10,
left: -20, bottom: 0 }}>
                <XAxis dataKey="time" axisLine={false} tickLine={false}
tick={{ fontSize: 12, fill: '#9CA3AF' }} dy={10} />
                <YAxis axisLine={false} tickLine={false} tick={{ fontSize: 12,
fill: '#9CA3AF' }} />
                <Tooltip
                  cursor={{ fill: '#F3F4F6' }}
                  contentStyle={{ borderRadius: '12px', border: 'none',
boxShadow: '0 4px 6px -1px rgb(0 0 0 / 0.1)' }}
                />
                <Bar dataKey="orders" fill="#3B82F6" radius={[4, 4, 0, 0]}
barSize={40} />
              </BarChart>
            </ResponsiveContainer>
          </div>
         ) : (
          <div className="h-72 w-full flex items-center justify-center
text-gray-400 text-sm">
            No orders data found for this date.
          </div>
         )}
        </div>

        {/* Top Selling Items Pie Chart */}
        <div className="bg-white rounded-3xl shadow-sm border
border-gray-100 p-6">
          <h3 className="text-lg font-bold mb-6 flex items-center
space-x-2">
            <PieChartIcon className="w-5 h-5 text-[#FF6321]" />
            <span>Top Selling Items</span>
          </h3>
          {topItems.length > 0 ? (
            <div className="h-72 w-full flex flex-col justify-center items-
center">
```

```
            <ResponsiveContainer width="100%" height="100%">
              <PieChart>
                <Pie
                  data={topItems}
                  cx="50%"
                  cy="50%"
                  innerRadius={60}
                  outerRadius={90}
                  paddingAngle={5}
                  dataKey="value"
                >
                  {topItems.map((entry, index) => (
                    <Cell key={`cell-${index}`} fill={COLORS[index %
COLORS.length]} />
                  ))}
                </Pie>
                <Tooltip
                  contentStyle={{ borderRadius: '12px', border: 'none',
boxShadow: '0 4px 6px -1px rgb(0 0 0 / 0.1)' }}
                  formatter={(value, name) => [`${value} sold`, name]}
                />
                <Legend iconType="circle" wrapperStyle={{ fontSize:
'12px' }} />
              </PieChart>
            </ResponsiveContainer>
          </div>
        ) : (
          <div className="h-72 w-full flex items-center justify-center
text-gray-400 text-sm">
            No items sold on this date.
          </div>
        )}
      </div>
    </div>
  )}
```

```
    </div>
  );
}
```

13.BIBLIOGYAPHY

The development of this project was supported by various online resources and learning platforms. These sources helped in understanding concepts, technologies, and implementation techniques.

- Google – for research, problem-solving, and concept understanding
- YouTube – for tutorials and practical guidance on React, Node.js, and project development
- Official Documentation – for detailed information on technologies like React, Node.js, Express, and SQLite